



**UNIVERSIDAD DE LA
CUENCA DEL PLATA**

ACTIVIDAD EVALUATIVA 2

CARRERA: Ingeniería en Sistemas de Información

MATERIA: Paradigmas y Lenguajes de Programación II

PROFESORES: Jose A. Fernandez, Silvia Romina Stacciuoli

ESTUDIANTE: Azcona Valentina

FECHA: 5/10/2026

Introducción

La Actividad de Evaluación 2 propone construir una biblioteca de clases capaz de simular de manera discreta y determinista la administración de procesos, memoria y CPU. El sistema se desarrolla en TypeScript y se verifica mediante pruebas automatizadas con Vitest. No se utiliza interfaz gráfica, menú de consola, función main ni temporizadores reales; el comportamiento se observa a través de métodos consultables y tests.

Objetivo y escenario

El escenario representa un laboratorio universitario que necesita estudiar cómo varios procesos comparten una memoria limitada y una única CPU. Cada proceso solicita una cantidad de memoria, puede esperar hasta obtener un bloque contiguo, pasa a la cola de Listos, consume CPU mediante Round Robin, puede bloquearse por un evento determinista de Entrada/Salida y libera su memoria al finalizar.

La simulación comienza en el tick 0, con la CPU libre, las colas vacías y la memoria representada por un único bloque libre. La memoria total, el quantum y la política de asignación son configurables. Como referencia de la consigna se utilizan 1024 KB y quantum 2, aunque los tests emplean distintas configuraciones para verificar casos normales, límites y errores.

Diseño y responsabilidades de las clases

- **Proceso:** Protege PID, memoria requerida, CPU restante, quantum consumido, estado y bloqueo. Centraliza las transiciones válidas de estado y la lógica asociada a eventos de E/S.
- **BloqueMemoria:** representa una partición contigua de RAM mediante inicio, tamaño, condición de libre/ocupado y PID asociado.
- **Memoria:** Mantiene el mapa de bloques contiguos, asigna y libera memoria, divide bloques y realiza coalescencia. Preserva orden, continuidad y tamaño total.
- **Planificador:** Administra la CPU y la cola FIFO de Listos. Implementa despacho, quantum, rotación y conteo de cambios de contexto.
- **Simulacion:** Coordina las fases de cada tick y expone una API pública reducida para registrar procesos, programar E/S, avanzar la simulación y consultar resultados.
- **Estrategias de asignación:** PrimerAjuste, MejorAjuste y PeorAjuste implementan IPoliticaAsignacion y encapsulan el criterio de selección del bloque.

Decisiones de Programación Orientada a Objetos

La principal decisión de diseño fue distribuir el comportamiento entre objetos con responsabilidades específicas, evitando una clase única que concentrara toda la lógica. El dominio se modela mediante clases con estado privado y operaciones controladas. Las reglas relevantes se implementan dentro del objeto responsable:

- Proceso protege las transiciones y contadores
- Memoria preserva las invariantes de los bloques
- Planificador administra CPU y cola
- Simulacion coordina las fases.

Además:

- Se evita la exposición de atributos públicos modificables sin control.
- Las operaciones internas de admisión, reintento y finalización permanecen encapsuladas en Simulacion.
- Las consultas devuelven copias o vistas de solo lectura para impedir que el código cliente altere el estado interno.
- No se introducen jerarquías de herencia artificiales; se prefiere composición cuando representa correctamente la relación entre objetos.
- Las estrategias de asignación permiten variar el comportamiento sin duplicar la administración de memoria.

Interfaces, abstracción, composición y encapsulamiento

El proyecto utiliza interfaces para expresar contratos entre componentes y reducir el acoplamiento. Entre las principales se encuentran IProceso, IMemoria, IPlanificador, ISimulacion, IProcesoVista, IBloqueMemoria, IEstadoSistema, IMetricas e IPoliticaAsignacion.

- Abstracción: los consumidores trabajan con contratos y vistas que exponen únicamente la información necesaria.
- Polimorfismo: IPoliticaAsignacion permite utilizar PrimerAjuste, MejorAjuste o PeorAjuste mediante el mismo método seleccionar().
- Composición: Simulacion contiene y coordina Memoria y Planificador; Memoria administra una colección de bloques.
- Encapsulamiento: Proceso protege transiciones y contadores; Memoria no expone sus bloques internos de forma modificable; Simulacion protege sus colas y operaciones internas.
- Principios SOLID: se busca responsabilidad única, bajo acoplamiento y extensión mediante estrategias sin mezclar criterios de asignación con otras responsabilidades.

Política de memoria utilizada

La memoria se administra mediante asignación contigua y bloques ordenados por dirección. El proyecto implementa tres políticas seleccionables: First-Fit, Best-Fit y Worst-Fit. First-Fit toma el primer bloque suficiente; Best-Fit el bloque suficiente de menor tamaño; Worst-Fit el bloque suficiente de mayor tamaño. Ante empates se conserva la menor dirección. Si el bloque elegido es mayor que la solicitud, se divide; si el ajuste es exacto, no se genera un bloque de tamaño cero.

Si la suma de memoria libre alcanza pero ningún hueco individual es suficientemente grande, la asignación falla sin modificar el mapa. Al finalizar un proceso se libera su bloque

y se fusionan automáticamente los bloques libres adyacentes. Esta coalescencia no desplaza bloques ocupados y no constituye compactación.

La política por defecto es First-Fit, aunque la configuración permite seleccionar Best-Fit o Worst-Fit. Se agregó una prueba comparativa con el mismo escenario para demostrar que cada estrategia elige un hueco diferente según su criterio.

Estados de los procesos y orden de ejecución de cada tick

Los estados implementados son: Nuevo, Esperando Memoria, Listo, Ejecutando, Bloqueado y Terminado.

El flujo normal es: Nuevo → Esperando Memoria → Listo → Ejecutando.

Desde Ejecutando, el proceso puede volver a Listo por agotamiento de quantum, pasar a Bloqueado por E/S o pasar a Terminado cuando la CPU restante llega a cero. Desde Bloqueado vuelve a Listo cuando termina el temporizador. Terminado es un estado final.

Cada llamada a avanzarTick() ejecuta exactamente las siguientes fases:

- Admisión de procesos e intento de asignación de memoria.
- Actualización de procesos bloqueados.
- Despacho y ejecución Round Robin.
- Actualización del reloj y métricas.

Como máximo un proceso consume una unidad de CPU por tick. La memoria liberada por una finalización en el paso de CPU queda disponible para procesos en espera recién en la fase de admisión del tick siguiente. Un proceso desbloqueado, en cambio, puede volver a Listos y ser despachado en ese mismo tick.

Matriz RF01 a RF10 de clases, métodos y tests

RF	Clase / método principal	Tests asociados
RF01	Simulacion.constructor	simulacion.test.ts
RF02	Simulacion.registrarProceso(), Proceso	proceso.test.ts
RF03	Simulacion.admitirProceso(), reintento y estados de Proceso	estados.test.ts
RF04	Memoria.asignar(), estrategias IPoliticaAsignacion	memoria.test.ts; politicasAsignacion.test.ts
RF05	Memoria.liberar() y coalescencia	memoria.test.ts
RF06	Simulacion.avanzarTick()	ordenTick.test.ts

RF07	Planificador y Round Robin	planificador.test.ts; roundRobin.test.ts
RF08	Proceso.programarEventoES(, bloquearPorES(), actualizarBloqueo())	entradaSalida.test.ts
RF09	Simulacion.consultarMetricas()	metricas.test.ts
RF10	Simulacion.consultarEstado()	estadoSistema.test.ts

Resultados y cobertura

La versión final verificada del proyecto presenta los siguientes resultados:

Indicador	Resultado
Suites de prueba	11
Tests aprobados	76 / 76
Typecheck	Sin errores
Statements	94.54%
Branches	91.46%
Functions	97.64%
Lines	94.54%

La cobertura global de líneas supera el mínimo exigido de 90%. El comando reproducible utilizado es: npm run coverage.

[Insertar captura aquí]

Captura sugerida: ejecución de npm test con 76/76 tests aprobados.

[Insertar captura aquí]

Captura sugerida: reporte de cobertura con 94.54% de líneas.

Análisis de fragmentación externa

La fragmentación externa aparece cuando la memoria libre total está distribuida en varios huecos no contiguos. En ese caso puede existir memoria libre suficiente en total, pero una solicitud puede no entrar porque necesita un único bloque contiguo.

La métrica utilizada es: Fragmentación externa = $100 \times (1 - \text{mayor bloque libre} / \text{memoria libre total})$. Si la memoria libre total es 0, la fragmentación se define como 0%.

Caso mínimo verificado por tests: existen huecos no contiguos de 100 KB y 300 KB. La memoria libre total es 400 KB y el mayor bloque libre es 300 KB. Por lo tanto: $100 \times (1 - 300/400) = 25\%$ de fragmentación externa.

La coalescencia reduce la fragmentación cuando una liberación deja dos o más bloques libres adyacentes, ya que esos bloques se fusionan en uno mayor. Sin embargo, no puede unir huecos separados por bloques ocupados porque no existe compactación.

La comparación de políticas demuestra que la elección del hueco modifica la distribución residual. First-Fit privilegia la primera dirección disponible; Best-Fit minimiza el sobrante inmediato pero puede crear huecos pequeños; Worst-Fit utiliza el mayor hueco y conserva un residuo grande. Ninguna estrategia es universalmente superior: su efecto depende del patrón de solicitudes y liberaciones.

Diagrama de clases

El diagrama de clases corresponde al código final y muestra las clases Proceso, BloqueMemoria, Memoria, Planificador, Simulacion y las estrategias de asignación, junto con sus interfaces y relaciones de realización/composición.

Versión legible: ([docs/AE2_diagrama_clases.drawio.png](#))

(Y en la carpeta Drive)

Versión editable: ([docs/AE2_diagrama_clases.drawio.png](#))

(Y en la carpeta Drive)

Diagramas de secuencia

Los diagramas de secuencia representan los principales flujos de colaboración entre objetos y permiten observar el orden de las operaciones.

- Secuencia 1: registro, admisión y asignación de memoria (RF02, RF03 y RF04).

Editable: ([docs/Diagramas/AE2_diagramas_secuencia%201.drawio](#))

Imágen: ([docs/Diagramas/AE2_diagramas_secuencia-Secuencia%201%20\(RF02.%20RF03.%20RF04\).drawio.png](#))

- Secuencia 2: avance de tick y planificación Round Robin (RF06 y RF07).

Editable: ([docs/Diagramas/AE2_diagramas_secuencia%202.drawio](#))

Imágen:

([docs/Diagramas/AE2_diagramas_secuencia-Secuencia%202%20\(RF06.%20RF07\).drawio.png](#))

- Secuencia 3: Entrada/Salida y métricas (RF08 y RF09).

Editable: ([docs/Diagramas/AE2_diagramas_secuencia%203.drawio](#))

Imágen: ([docs/Diagramas/AE2_diagramas_secuencia-Secuencia%203%20\(RF08.%20RF09\).drawio.png](#))

Conclusiones

El desarrollo permitió aplicar Programación Orientada a Objetos sobre un dominio con reglas concretas. La separación de responsabilidades, el encapsulamiento, las interfaces y el polimorfismo para las políticas de asignación permiten mantener el modelo comprensible y verificable. La protección de transiciones e invariantes evita que el estado pueda alterarse

por fuera de las reglas del dominio, mientras que las pruebas automatizadas ofrecen evidencia reproducible del comportamiento.